

# Step 4 — A Verifier Agent That Catches the Worker's Bad Answers — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume `agents/worker_agent.py` already exports `run_worker(task, max_iterations) -> AgentResult` — Step 3's Worker, moved into `agents/` and renamed per the README's final file layout, with `run_agent` renamed to `run_worker`.

## Basic Version

### Approach 1 — an agreeable Verifier (works, but proves nothing)

```
# agents/verifier_agent.py
from pydantic import BaseModel
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI

class VerifierVerdict(BaseModel):
    approved: bool
    reason: str

def verify(question, worker_answer):
    model = ChatOpenAI(model="gpt-4o-mini").with_structured_output(VerifierVerdict)
    prompt = ChatPromptTemplate.from_messages([
        ("system", "Review this answer and note any issues."),
        ("human", "Question: {question}\n\nProposed answer: {answer}"),
    ])
    chain = prompt | model
    return chain.invoke({"question": question, "answer": worker_answer})
```

**Expected output**, run against a Worker answer that's actually wrong (say, a stale weather lookup presented as current):

```
approved=True reason='The answer addresses the question and appears reasonable.'
```

This is the exact failure the README warns about — "Review this answer and note any issues" reads as a light proofread, not a real check, and the model defaults to approving anything that merely *sounds* plausible. It's a real, independently-running second model call, which is the minimum bar for "2 agents" — but it isn't actually catching anything.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Intermediate Version

### Approach 1 — a real, structured verdict and a wired-up main flow

```
# agents/verifier_agent.py
from pydantic import BaseModel
from langchain_core.prompts import ChatPromptTemplate
from langchain_openai import ChatOpenAI
from logging_setup import get_logger

logger = get_logger(__name__)

VERIFIER_SYSTEM_PROMPT = """You are an independent verifier. You did not produce
Given a question and a proposed answer, judge two things:
(1) does the answer actually address the question that was asked?
(2) does it look consistent with real tool use, rather than an unsupported guess?

class VerifierVerdict(BaseModel):
    approved: bool
    reason: str

def verify(question: str, worker_answer: str) -> VerifierVerdict:
    model = ChatOpenAI(model="gpt-4o-mini").with_structured_output(VerifierVerdict)
    prompt = ChatPromptTemplate.from_messages([
        ("system", VERIFIER_SYSTEM_PROMPT),
        ("human", "Question: {question}\n\nProposed answer: {answer}"),
    ])
    chain = prompt | model
    verdict = chain.invoke({"question": question, "answer": worker_answer})
    logger.info("Verifier verdict: approved=%s reason=%s", verdict.approved, verdict.reason)
    return verdict
```

```
# main.py
from agents.worker_agent import run_worker
from agents.verifier_agent import verify
from logging_setup import get_logger

logger = get_logger(__name__)
```

```
task = "What's the current weather in Paris?"
result = run_worker(task, max_iterations=6)
logger.info("Worker answer: %s", result.answer)

verdict = verify(task, result.answer)
if verdict.approved:
    print(result.answer)
else:
    print(f"FLAGGED: {verdict.reason}")
```

**Expected output**, on a genuinely good Worker answer:

```
It is sunny and 72F in Paris.
```

**Difference from Basic:** the system prompt now names 2 concrete, checkable questions instead of an open-ended "note any issues" — closer to a real check, but still an "approve unless clearly wrong" default, which is exactly what makes a Verifier pass borderline cases too easily. Logging is wired in for both agents separately, as the README's Step 4 checklist asks.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

## Advanced Version

### Approach 1 — rejection-biased prompt, plus a bounded retry loop

```
# agents/verifier_agent.py (system prompt sharpened)
VERIFIER_SYSTEM_PROMPT = """You are an independent verifier. You did not produce
and you should not assume it is correct just because it sounds fluent or confident.

Given a question and a proposed answer, judge two things:
(1) does the answer actually address the question that was asked?
(2) does it look consistent with real tool use, rather than an unsupported guess?

If you cannot confirm the answer is well-supported, REJECT it. Do not give the
benefit of the doubt – a false approval is worse than a false rejection here."""
```

```
# main.py
from agents.worker_agent import run_worker
from agents.verifier_agent import verify
from logging_setup import get_logger

logger = get_logger(__name__)

def run_worker_with_verification(task: str, max_verify_attempts: int = 2):
    result = run_worker(task, max_iterations=6)

    for attempt in range(1, max_verify_attempts + 1):
        verdict = verify(task, result.answer)
        logger.info("Verify attempt %d/%d: approved=%s", attempt, max_verify_attempts, verdict.approved)

        if verdict.approved:
            return result.answer, verdict

    if attempt == max_verify_attempts:
        return None, verdict # exhausted retries – caller shows the flag, no need to log

    retry_task = f"{task}\n\nA reviewer rejected your last answer: {verdict.reason}. Please try again."
    result = run_worker(retry_task, max_iterations=6)

    raise AssertionError("unreachable") # the loop always returns inside max_verify_attempts
```

```

answer, verdict = run_worker_with_verification("What's the current weather in Paris?")
if answer is not None:
    print(answer)
else:
    print(f"Could not get a verified answer after retries. Last flag: {verdict.reason}")

```

**Expected output**, when the Worker's first attempt gets flagged and the retry succeeds:

```
It is sunny and 72F in Paris.
```

**Expected output**, when a task is engineered to fail verification twice in a row (like asking about something no available tool can actually answer):

```
Could not get a verified answer after retries. Last flag: The answer claims a specific date but no tool was ever used to check it.
```

## Approach 2 — a test that proves the Verifier can actually reject

```

# test_agent.py (excerpt)
def test_verifier_rejects_unsupported_answer():
    # deliberately construct a "Worker answer" that was never actually produced
    # by a tool call, to confirm the Verifier catches it and isn't a rubber stamp
    question = "What is the current exchange rate from USD to EUR?"
    fabricated_answer = "The current rate is 1 USD = 0.93 EUR." # no tool was ever used to check it

    verdict = verify(question, fabricated_answer)

    assert verdict.approved is False

```

**Expected output:** the test passes — the rejection-biased prompt, given an answer with a suspiciously specific number and no supporting tool-use context to check it against, declines to approve rather than assuming a plausible-sounding number must be correct.

**Difference from Intermediate, and between these 2 Advanced approaches:** Intermediate's Verifier can still be talked into approving a fluent-but-unsupported answer, because "approve unless clearly wrong" is its actual default even with 2 named checks. Approach 1 flips that default and adds a bounded retry loop, so a genuine rejection leads somewhere (another real attempt, then a flag) instead of the flow just stopping. Approach 2 doesn't change the agents at all — it's the concrete proof, required by the README's own checklist, that the Verifier catches at least one real bad case instead of always approving.

**Which one should you actually write?** All of Approach 1, and keep Approach 2's test (or one like it) in `test_agent.py` permanently. A Verifier is only worth the extra API call if it's biased toward catching problems — an agreeable one adds cost and latency for the appearance of safety with none of the actual benefit. The bounded retry (2 attempts, then flag) matters just as much: without it, a genuinely bad task can bounce between Worker and Verifier exactly the way an un-limited Step 2 loop could run forever — the fix is the same idea, one level up.